

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Online Shopping Mart</title>
  <!-- Tailwind CSS CDN -->
  <script src="https://cdn.tailwindcss.com"></script>
  <style>
    @import
url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700;8
00&display=swap');
    body {
      font-family: 'Inter', sans-serif;
      background-color: #f3f4f6;
    }
    .product-card {
      transition: transform 0.3s, box-shadow 0.3s;
      background-color: white;
      border-radius: 12px;
      overflow: hidden;
      box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px
-2px rgba(0, 0, 0, 0.06);
    }
    .product-card:hover {
      transform: translateY(-4px);
      box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px
-4px rgba(0, 0, 0, 0.1);
    }
    /* Mobile-first sticky footer for Cart button */
    #cart-button-mobile {
      position: fixed;
      bottom: 0;
      left: 0;
      right: 0;
      z-index: 50;
      padding: 1rem;
      box-shadow: 0 -5px 10px rgba(0, 0, 0, 0.1);
      background-color: white;
    }
    /* Style for the right-side fixed cart panel on desktop */
    @media (min-width: 1024px) {
      #cart-panel {
        position: fixed;
        top: 0;
        right: 0;
        height: 100vh;

```

```

        width: 384px; /* w-96 */
        overflow-y: auto;
        transform: translateX(0); /* Always visible on large
screens */
        transition: none;
    }
}
/* Full-screen cart panel on mobile */
@media (max-width: 1023px) {
    #cart-panel {
        position: fixed;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        transform: translateX(100%); /* Hidden by default */
        transition: transform 0.3s ease-out;
        z-index: 100;
    }
    #cart-panel.open {
        transform: translateX(0);
    }
}
</style>
</head>
<body class="bg-gray-50">

    <div id="app" class="min-h-screen">
        <!-- Header: Updated for Auth Buttons -->
        <header class="bg-white shadow-md sticky top-0 z-40">
            <div class="max-w-7xl mx-auto p-4 flex justify-between
items-center">
                <h1 class="text-2xl font-extrabold
text-indigo-600">eMart  </h1>
                <div class="flex items-center space-x-3">
                    <!-- Visible when LOGGED IN -->
                    <div id="logged-in-status" class="hidden
items-center space-x-3">
                        <span id="user-email-display" class="text-sm
text-gray-700 font-medium truncate max-w-[150px]
sm:max-w-none">Loading...</span>
                        <button onclick="logoutUser()"
class="bg-red-500 hover:bg-red-600 text-white text-sm font-semibold
py-2 px-3 rounded-lg shadow-md transition duration-150">
                            Logout
                        </button>
                    </div>
                    <!-- Visible when LOGGED OUT -->

```

```

        <button id="login-register-btn"
onclick="openAuthModal('login')" class="bg-indigo-600
hover:bg-indigo-700 text-white text-sm font-semibold py-2 px-3
rounded-lg shadow-md transition duration-150">
            Login / Register
        </button>
    </div>
</div>
</header>

<main class="max-w-7xl mx-auto lg:flex">
    <!-- Product Grid Area -->
    <div id="product-grid-area" class="p-4 lg:p-8 lg:w-3/4">
        <h2 class="text-3xl font-bold text-gray-800
mb-6">Featured Products (Aapki Pasand)</h2>

        <div id="product-grid" class="grid grid-cols-2
md:grid-cols-3 gap-4 md:gap-6">
            <!-- Products will be dynamically inserted here
-->

        </div>
    </div>

    <!-- Cart Panel (Desktop View) -->
    <div id="cart-panel" class="bg-gray-100 lg:w-96 p-6
lg:border-l border-gray-200">
        <div class="flex flex-col">
            <div class="flex justify-between items-center
mb-4">

                <h3 class="text-2xl font-bold
text-gray-800">Your Cart </h3>
                <!-- Close button for mobile -->
                <button id="close-cart-btn" class="lg:hidden
text-gray-600 hover:text-gray-900 focus:outline-none"
onclick="toggleCartPanel()">

                    <svg class="w-6 h-6" fill="none"
stroke="currentColor" viewBox="0 0 24 24"
xmlns="http://www.w3.org/2000/svg"><path stroke-linecap="round"
stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 6l12
12"></path></svg>

                </button>
            </div>
            <!-- Clear All Cart Items Button -->
            <button id="clear-cart-btn" onclick="clearCart()"
class="text-sm text-red-500 hover:text-red-700 font-semibold text-left
mb-4 transition duration-150 disabled:opacity-50" disabled>
                Clear All Items
            </button>
        </div>
    </div>
</main>

```

```

        </div>

        <div id="cart-items" class="space-y-4 h-auto
lg:h-[calc(100vh-250px)] overflow-y-auto pr-2">
            <!-- Cart items will be dynamically inserted here
-->

            <p id="empty-cart-message" class="text-gray-500
text-center py-8">Aapki Cart Khali Hai.</p>
        </div>

        <!-- Cart Summary -->
        <div id="cart-summary" class="mt-6 pt-4 border-t
border-gray-300">
            <div class="flex justify-between items-center
text-xl font-semibold text-gray-800 mb-4">
                <span>Total:</span>
                <span id="cart-total">₹ 0.00</span>
            </div>

            <button onclick="handleCheckout()"
id="checkout-btn" class="w-full bg-green-600 hover:bg-green-700
text-white font-bold py-3 rounded-xl shadow-lg transition duration-150
disabled:bg-gray-400" disabled>
                Proceed to Checkout
            </button>
        </div>
    </div>
</main>

    <!-- Mobile Floating Cart Button -->
    <div id="cart-button-mobile" class="lg:hidden">
        <button onclick="toggleCartPanel()" class="w-full
bg-indigo-600 hover:bg-indigo-700 text-white font-bold py-3 rounded-xl
shadow-lg flex items-center justify-center">
            <span id="mobile-cart-count" class="mr-2 text-sm
bg-white text-indigo-600 px-2 rounded-full font-extrabold">0</span>
            View Cart & Checkout
        </button>
    </div>
</div>

    <!-- ----- -->
    <!-- --- Authentication Modal (Login/Register) --- -->
    <!-- ----- -->
    <div id="auth-modal" class="fixed inset-0 bg-gray-900
bg-opacity-70 flex items-center justify-center p-4 z-[150] hidden">
        <div class="bg-white rounded-xl p-6 w-full max-w-md
shadow-2xl" onclick="event.stopPropagation()">

```

```

        <div class="flex justify-between items-center mb-6">
            <h3 id="auth-modal-title" class="text-2xl font-bold
text-gray-800">Login</h3>
            <button onclick="closeAuthModal()"
class="text-gray-400 hover:text-gray-600">
                <svg class="w-6 h-6" fill="none"
stroke="currentColor" viewBox="0 0 24 24"
xmlns="http://www.w3.org/2000/svg"><path stroke-linecap="round"
stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 6l12
12"></path></svg>
            </button>
        </div>

        <form id="auth-form" onsubmit="event.preventDefault();
handleAuthSubmit()">
            <div class="mb-4">
                <label for="email" class="block text-sm
font-medium text-gray-700 mb-1">Email</label>
                <input type="email" id="auth-email" required
class="w-full p-3 border border-gray-300 rounded-lg
focus:ring-indigo-500 focus:border-indigo-500"
placeholder="user@example.com">
            </div>
            <div class="mb-6">
                <label for="password" class="block text-sm
font-medium text-gray-700 mb-1">Password</label>
                <input type="password" id="auth-password" required
minlength="6" class="w-full p-3 border border-gray-300 rounded-lg
focus:ring-indigo-500 focus:border-indigo-500" placeholder="Minimum 6
characters">
            </div>

            <p id="auth-error" class="text-red-500 text-sm mb-4
hidden"></p>

            <button type="submit" id="auth-submit-btn"
class="w-full bg-indigo-600 hover:bg-indigo-700 text-white font-bold
py-3 rounded-xl transition duration-150">
                Login
            </button>
        </form>

        <div class="mt-4 text-center">
            <button id="toggle-auth-mode-btn"
onclick="toggleAuthMode()" class="text-sm text-indigo-600
hover:text-indigo-800 font-medium">
                New user? Create an account.
            </button>

```

```
        </div>
    </div>
</div>
```

```
<!-- Firebase Core Libraries -->
<script type="module">
    import { initializeApp } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-app.js";
    import { getAuth, onAuthStateChanged,
createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut }
from "https://www.gstatic.com/firebasejs/11.6.1/firebase-auth.js";
    import { getFirestore, doc, setDoc, onSnapshot } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-firestore.js";
    import { setLogLevel } from
"https://www.gstatic.com/firebasejs/11.6.1/firebase-firestore.js";

    // Global variables provided by the Canvas environment
    const appId = typeof __app_id !== 'undefined' ? __app_id :
'default-shopping-app-id';
    const firebaseConfig = typeof __firebase_config !==
'undefined' ? JSON.parse(__firebase_config) : null;

    let db;
    let auth;
    let userId = null;
    let cartRef = null;
    let cartItems = {}; // {productId: {itemData, quantity}}
    let unsubscribeCart = null; // To store the onSnapshot
listener function

    // Auth Modal State
    let isRegisterMode = false;

    // Product data (Mock data for the shop)
    const PRODUCTS = [
        { id: 'p001', name: 'Smartwatch V3', price: 8999,
imageUrl:
'https://placeholder.co/400x400/818CF8/ffffff?text=Smart+Watch',
description: 'Advanced fitness tracking and long battery life.' },
        { id: 'p002', name: 'Wireless Earbuds Pro', price: 3499,
imageUrl: 'https://placeholder.co/400x400/EC4899/ffffff?text=Earbuds',
description: 'Active noise cancellation and crystal-clear audio.' },
        { id: 'p003', name: 'Portable Power Bank 20K', price:
1999, imageUrl:
'https://placeholder.co/400x400/10B981/ffffff?text=Power+Bank',
description: 'Fast charging for all your devices.' },
        { id: 'p004', name: 'Gaming Mouse', price: 1499, imageUrl:
```

```

    'https://placeholder.co/400x400/F59E0B/ffffff?text=Mouse', description:
    'Ergonomic design with adjustable DPI.' },
    { id: 'p005', name: 'LED Desk Lamp', price: 1299,
    imageUrl: 'https://placeholder.co/400x400/3B82F6/ffffff?text=Desk+Lamp',
    description: 'Dimmable with multiple color temperature modes.' },
    { id: 'p006', name: 'Premium Espresso Maker', price:
    18999, imageUrl:
    'https://placeholder.co/400x400/D97706/ffffff?text=Coffee+Machine',
    description: 'Brew café-quality espresso at home.' }
  ];

```

```

    // Expose functions globally for HTML access
    window.addToCart = addToCart;
    window.removeFromCart = removeFromCart;
    window.handleCheckout = handleCheckout;
    window.toggleCartPanel = toggleCartPanel;
    window.clearCart = clearCart;
    window.openAuthModal = openAuthModal;
    window.closeAuthModal = closeAuthModal;
    window.handleAuthSubmit = handleAuthSubmit;
    window.toggleAuthMode = toggleAuthMode;
    window.logoutUser = logoutUser;

    /**
     * Initializes Firebase and sets up the authentication state
    listener.
     */
    function initFirebase() {
      try {
        if (!firebaseConfig) {
          console.error("Firebase config not found. Cannot
    initialize Firestore.");
          return;
        }

        // setLogLevel('Debug'); // Enable for debugging
        const app = initializeApp(firebaseConfig);
        auth = getAuth(app);
        db = getFirestore(app);

        // Set up Auth State Listener
        onAuthStateChanged(auth, (user) => {
          updateAuthState(user);
        });

        renderProductGrid();

      } catch (error) {

```

```

        console.error("Firebase initialization failed:",
error);
    }
}

/**
 * Updates UI and data listeners based on the Firebase Auth
state.
 * @param {object} user - The current Firebase User object or
null.
 */
function updateAuthState(user) {
    const loginRegisterBtn =
document.getElementById('login-register-btn');
    const loggedInStatus =
document.getElementById('logged-in-status');
    const userEmailDisplay =
document.getElementById('user-email-display');

    // 1. Clean up old listener if it exists
    if (unsubscribeCart) {
        unsubscribeCart();
        unsubscribeCart = null;
    }

    // 2. Clear current cart data and UI
    userId = null;
    cartItems = {};
    cartRef = null;
    renderCart(); // Clear the UI instantly

    if (user) {
        // LOGGED IN
        userId = user.uid;
        const email = user.email || "Guest User";
        userEmailDisplay.textContent = email;

        loggedInStatus.classList.remove('hidden');
        loggedInStatus.classList.add('flex');
        loginRegisterBtn.classList.add('hidden');

        // Define the private cart path for the logged-in user
        // Data stored as:
artifacts/{appId}/users/{userId}/shopping_data/cart
        cartRef = doc(db,
`artifacts/${appId}/users/${userId}/shopping_data/cart`);
        setupRealtimeCartListener();
    }
}

```



```

        console.log("User logged in:", userId);

    } else {
        // LOGGED OUT
        userId = crypto.randomUUID(); // Use a temp ID for
anonymous shopping session

        loggedInStatus.classList.add('hidden');
        loggedInStatus.classList.remove('flex');
        loginRegisterBtn.classList.remove('hidden');

        // Define a temporary path for the anonymous user
        cartRef = doc(db,
`artifacts/${appId}/users/${userId}/shopping_data/temp_cart`);
        setupRealtimeCartListener();

        console.log("User logged out. Using temp ID:",
userId);
    }
}

/**
 * Handles the form submission for Login or Register.
 */
async function handleAuthSubmit() {
    const email = document.getElementById('auth-email').value;
    const password =
document.getElementById('auth-password').value;
    const errorElement =
document.getElementById('auth-error');

    errorElement.classList.add('hidden');
    document.getElementById('auth-submit-btn').textContent =
"Processing...";
    document.getElementById('auth-submit-btn').disabled =
true;

    try {
        if (isRegisterMode) {
            await createUserWithEmailAndPassword(auth, email,
password);
        } else {
            await signInWithEmailAndPassword(auth, email,
password);
        }
        closeAuthModal();
        showCustomMessage("Success!", isRegisterMode ?
"Registration successful! Welcome to eMart." : "Login successful! Your

```

```

cart is now linked to your account.");

    } catch (error) {
        let errorMessage = "An unknown error occurred.";
        console.error("Auth Error:", error);

        switch (error.code) {
            case 'auth/invalid-email':
                errorMessage = "Invalid email format."; break;
            case 'auth/user-disabled':
                errorMessage = "This user account has been
disabled."; break;
            case 'auth/user-not-found':
            case 'auth/wrong-password':
                errorMessage = "Invalid login credentials.";
break;
            case 'auth/email-already-in-use':
                errorMessage = "This email is already
registered."; break;
            case 'auth/weak-password':
                errorMessage = "Password should be at least 6
characters."; break;
            default:
                errorMessage = "Authentication failed. Please
check your credentials.";
        }
        errorElement.textContent = errorMessage;
        errorElement.classList.remove('hidden');

    } finally {
        document.getElementById('auth-submit-btn').textContent
= isRegisterMode ? "Register" : "Login";
        document.getElementById('auth-submit-btn').disabled =
false;
    }
}

/**
 * Logs the current user out.
 */
async function logoutUser() {
    try {
        await signOut(auth);
        showCustomMessage("Logged Out", "You have been
successfully logged out.");
    } catch (error) {
        console.error("Logout Error:", error);
        showCustomMessage("Error", "Could not log out. Please

```

```

try again.");
    }
}

/**
 * Opens the authentication modal.
 * @param {string} mode - 'login' or 'register'.
 */
function openAuthModal(mode) {
    isRegisterMode = (mode === 'register');
    toggleAuthMode(isRegisterMode);

document.getElementById('auth-modal').classList.remove('hidden');
    document.getElementById('auth-email').focus();
}

/**
 * Closes the authentication modal.
 */
function closeAuthModal() {

document.getElementById('auth-modal').classList.add('hidden');

document.getElementById('auth-error').classList.add('hidden');
    document.getElementById('auth-form').reset();
}

/**
 * Toggles between Login and Register modes in the modal.
 */
function toggleAuthMode(setRegister = !isRegisterMode) {
    isRegisterMode = setRegister;
    const title = document.getElementById('auth-modal-title');
    const submitBtn =
document.getElementById('auth-submit-btn');
    const toggleBtn =
document.getElementById('toggle-auth-mode-btn');

    if (isRegisterMode) {
        title.textContent = "Create Account";
        submitBtn.textContent = "Register";
        toggleBtn.innerHTML = "Already have an account? Log
in.";
    } else {
        title.textContent = "Login";
        submitBtn.textContent = "Login";
        toggleBtn.innerHTML = "New user? Create an account.";
    }
}

```

```

document.getElementById('auth-error').classList.add('hidden');
}

/**
 * Renders all products in the main grid.
 */
function renderProductGrid() {
  const grid = document.getElementById('product-grid');
  grid.innerHTML = ''; // Clear existing content

  PRODUCTS.forEach(product => {
    const card = document.createElement('div');
    card.className = 'product-card flex flex-col
justify-between';
    card.innerHTML = `
      <div>
        
        <div class="p-4">
          <h4 class="text-lg font-bold
text-gray-800">${product.name}</h4>
          <p class="text-sm text-gray-500
mb-2">${product.description}</p>
          <p class="text-2xl font-extrabold
text-indigo-600">₹ ${product.price.toLocaleString('en-IN')}</p>
        </div>
      </div>
      <div class="p-4 pt-0">
        <button onclick="addToCart('${product.id}')"
class="w-full bg-indigo-500 hover:bg-indigo-600 text-white
font-semibold py-2 rounded-lg transition duration-150 ease-in-out">
          Add to Cart
        </button>
      </div>
    `;
    grid.appendChild(card);
  });
}

/**
 * Sets up the real-time listener for the user's shopping cart
data.
 */
function setupRealtimeCartListener() {
  if (!db || !cartRef) return;

```

```

// Unsubscribe from any previous listener before setting a
new one
if (unsubscribeCart) {
  unsubscribeCart();
}

unsubscribeCart = onSnapshot(cartRef, (docSnap) => {
  if (docSnap.exists() && docSnap.data().items) {
    // Firestore stores data as a simple map of items
    cartItems = docSnap.data().items;
  } else {
    cartItems = {};
  }
  // Always re-render the cart UI after data change
  renderCart();
}, (error) => {
  console.error("Error listening to cart data:", error);
});
}

/**
 * Saves the current cartItems object to Firestore.
 */
async function saveCart() {
  if (!cartRef) {
    console.error("Cart reference not set. Cannot save.");
    return;
  }
  try {
    // Save the cartItems map under the 'items' field
    await setDoc(cartRef, { items: cartItems }, { merge:
true });
  } catch (e) {
    console.error("Error saving cart: ", e);
  }
}

/**
 * Adds a product to the cart or increments its quantity.
 */
function addToCart(productId) {
  const product = PRODUCTS.find(p => p.id === productId);
  if (!product) return;

  if (cartItems[productId]) {
    cartItems[productId].quantity += 1;
  } else {

```

```

        cartItems[productId] = {
            itemData: product,
            quantity: 1
        };
    }
    saveCart(); // Save changes to Firestore
}

/**
 * Removes or decrements the quantity of a product from the
cart.
 */
function removeFromCart(productId, removeAll = false) {
    if (!cartItems[productId]) return;

    if (removeAll || cartItems[productId].quantity <= 1) {
        delete cartItems[productId];
    } else {
        cartItems[productId].quantity -= 1;
    }
    saveCart(); // Save changes to Firestore
}

/**
 * Clears all items from the cart.
 */
function clearCart() {
    if (Object.keys(cartItems).length > 0) {
        showCustomMessage("Cart Cleared", "Aapne saare items
cart se hata diye hain.");
        cartItems = {}; // Reset the cart
        saveCart(); // Save the empty cart to Firestore
    }
}

/**
 * Renders the cart contents and updates the total.
 */
function renderCart() {
    const cartPanel = document.getElementById('cart-items');
    const totalDisplay =
document.getElementById('cart-total');
    const checkoutBtn =
document.getElementById('checkout-btn');
    const mobileCount =
document.getElementById('mobile-cart-count');
    const emptyMessage =
document.getElementById('empty-cart-message');

```

```

    const clearCartBtn =
document.getElementById('clear-cart-btn');

    cartPanel.innerHTML = '';
    let total = 0;
    let count = 0;
    const itemsInCart = Object.values(cartItems);

    if (itemsInCart.length === 0) {
        emptyMessage.classList.remove('hidden');
        totalDisplay.textContent = '₹ 0.00';
        checkoutBtn.disabled = true;
        clearCartBtn.disabled = true;
        mobileCount.textContent = '0';
        cartPanel.appendChild(emptyMessage);
        return;
    }

    emptyMessage.classList.add('hidden');
    checkoutBtn.disabled = false;
    clearCartBtn.disabled = false;

    itemsInCart.forEach(item => {
        const itemTotal = item.itemData.price * item.quantity;
        total += itemTotal;
        count += item.quantity;

        const itemDiv = document.createElement('div');
        itemDiv.className = 'flex items-start bg-white p-3
rounded-lg shadow-sm border border-gray-200';
        itemDiv.innerHTML = `
            <img src="${item.

```